GEMINI

AGENT ADK

COURSE SLIDES · WSQ

Develop Multi AI Agent Applications with Gemini Agent ADK

WSQ Course Code: TGS-2024042961

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Version v1.3 · 12 August 2026

COURSE ADMINISTRATION

Welcome & Housekeeping

Digital Attendance (Mandatory)

1

Three times a day

Take the AM, PM and Assessment digital attendance — mandatory for every WSQ-funded course.

2

Trainer shows the QR

The trainer or administrator displays the digital attendance QR code from the SSG portal.

3

Scan and submit

Scan the QR code with your mobile phone camera and submit your attendance.

4

75% minimum

A minimum of 75% attendance is required to be eligible for assessment and funding.

About the Trainer

Your Trainer

General Trainer template —
to be completed by the trainer

NAME

TITLE / DESIGNATION

QUALIFICATIONS

AREAS OF EXPERTISE

TRAINING & INDUSTRY EXPERIENCE

CONTACT

YOUR TRAINER

About the Trainer

Dr Alfred Ang

Principal Trainer
Tertiary Infotech Academy Pte Ltd

ROLE
Principal Trainer, Tertiary Infotech Academy Pte Ltd

EXPERTISE
Generative AI, Large Language Models, agentic AI systems and applied machine learning.

DELIVERS
WSQ courses on AI, LLM application development, prompt engineering and data science.

FOUNDER
Founder and lead instructor at Tertiary Infotech / Tertiary Courses.

ICE-BREAKER

Let's Know Each Other

- Your name, organisation and role.
- Your experience with Python, APIs and Generative AI tools (if any).
- What you would like to automate or build with AI agents after this course.

HOUSEKEEPING

Ground Rules

1. Set your mobile phone to silent mode.
2. Participate actively — no question is too small.
3. Mutual respect: agree to disagree.
4. One conversation at a time.
5. Be punctual; return from breaks on time.
6. 75% attendance is required.

LMS / TMS · lms-tms.tertiaryinfotech.com

Download Your Course Material

1

1 · Go to the LMS portal

Open https://lms-tms.tertiaryinfotech.com in your browser.

2

2 · Log in

Sign in with the account e-mail you used to register for this course.

3

3 · Open this course

Select 'Develop Multi AI Agent Applications with Gemini Agent ADK'.

4

4 · Download

Download the Trainer Slides, Learner Guide and Lesson Plan (PDF).

5

5 · Get the labs

Clone the lab repository from GitHub — the link is on the course page.

6

6 · Keep them open

You may use these materials during the open-book assessment.

Skills Framework Alignment

1

TSC Title

Artificial Intelligence Application

2

TSC Code

AER-TEM-4026-1.1

3

A1 / A2

Analyse algorithms in AI applications; correlate algorithm design with efficiency.

4

A3 / A4

Identify and compare the strengths and limitations of AI applications.

5

A5

Assess the feasibility of AI applications for engineering processes.

6

A6

Assess the improvements AI applications bring to engineering processes.

SCHEDULE

Lesson Plan — 2 Days, 8 hours/day

Day 1

- Day 1 — Agentic AI foundations with Gemini ADK — first agents, tools, sessions and multi-agent handoff

– Digital Attendance (AM) · Introductions · Learning Outcomes

– Topic 1: Overview of Agentic AI in Gemini ADK (Labs 1–4)

– Lunch Break · Digital Attendance (PM)

– Topic 2: Build A Multi Agent App with Gemini ADK begins (Labs 5–8)

Day 2

- Day 2 — Multi-agent systems, agentic RAG and shipping an agent app with Streamlit

– Topic 2 continues (Labs 9–12)

– Topic 3: Build Agentic AI RAG in Gemini ADK (Labs 13–15)

– Topic 4: Agentic AI App with Streamlit (Labs 16–18)

– TRAQOM Survey · Digital Attendance (Assessment)

– Final Assessment (WA + PP)

WHAT YOU'LL ACHIEVE

Learning Outcomes

1

LO1 · Analyse LLM applications

Analyze the range of LLM applications using Generative AI and identify their industrial use cases.

2

LO2 · Establish Gemini designs

Establish Google Gemini GAI designs and assess improvements on engineering processes.

3

LO3 · Develop LLM applications

Develop LLM applications with the Agent Development Kit and assess their feasibility.

4

LO4 · Evaluate RAG

Evaluate the performance effectiveness of Retrieval Augmented Generation.

FOUR TOPICS · 18 HANDS-ON LABS

Course Outline

1

Topic 1 — Overview of Agentic AI in Gemini ADK

LLM & agentic AI foundations · the Gemini model family · ADK architecture · your first agent · tools.

2

Topic 2 — Build A Multi Agent App with Gemini ADK

Sessions & state · handoff · sub-agents · workflow agents · guardrails · structured output · MCP.

3

Topic 3 — Build Agentic AI RAG in Gemini ADK

The RAG pipeline · loading & splitting · embeddings · vector stores · retrieval · evaluation.

4

Topic 4 — Agentic AI App with Gemini ADK and Streamlit

YAML agents · Streamlit chat UI · async Runner · deployment and feasibility · capstone.

BEFORE YOU START

Briefing for Assessment

1

Do · Clear your desk

Place phones and other materials under the table or on the floor.

2

Don't · No recording

No photos or recording of assessment scripts.

3

Don't · No discussion

Work individually — no discussion during the assessment.

4

Do · Black or blue pen

Use a black or blue pen for hard-copy assessments.

5

Don't · No correction fluid

No liquid paper or correction tape may be used.

6

Do · Stop on time

Scripts are collected when time is up.

FINAL ASSESSMENT

Assessment

1

Written Assessment (WA)

Short-Answer Questions (SAQ) · 1 hour · open book. Tests the underpinning knowledge taught in the slides.

2

Practical Performance (PP)

Hands-on Gemini ADK agent build tasks · 1 hour · open book. Tests the ability you built in the labs.

3

Open book

You may use the course slides, the Learner Guide and approved materials only.

4

Eligibility

A minimum of 75% attendance is required to be eligible for assessment and funding.

5

Result

You are assessed as Competent (C) or Not Yet Competent (NYC) on each instrument.

6

Appeals

An appeal process is available if you wish to contest an assessment outcome.

ON ASSESSMENT DAY

Assessment Flow

1

TRAQOM survey

Scan the QR code on the…

2

Digital attendance

Scan the SSG assessment QR

3

Sit WA then PP

Open book · 1 hour each

4

Submit on the LMS

Upload your completed…

5

Sign the record

Sign the Assessment…

REMEMBER

All five steps are mandatory for WSQ funding — missing the digital attendance or the TRAQOM survey can invalidate your claim.

WSQ FUNDING

Criteria for Funding

1

Attendance

A minimum attendance rate of 75%, based on the SSG Digital Attendance record.

2

Assessment

Complete both assessment components and be assessed as 'Competent'.

3

Digital attendance

Scan the SSG QR code for AM, PM and Assessment on every training day.

4

TRAQOM survey

Complete the mandatory TRAQOM course feedback survey on the LMS.

LAB ENVIRONMENT

Set Up Before We Start

1

Google account

Sign in to Google AI Studio at https://aistudio.google.com to create a free API key.

2

Python 3.13

Install Python 3.13 and the uv package manager for the lab environment.

3

Lab repository

Clone the course labs from GitHub — the link is on your LMS course page.

4

Editor

Use VS Code or any editor you are comfortable with for editing agent.py files.

5

Optional keys

OpenWeather and Tavily free API keys are used by the tool-calling labs.

6

ADK docs

Bookmark the official ADK documentation at https://google.github.io/adk-docs/.

CORE CONCEPTS

From Large Language Models to AI Agents

What is a Large Language Model?

1

A model of language

An LLM such as Gemini, GPT-4 or Claude is trained to understand and generate human language.

2

Why 'large'

Large in both the volume of training data and the model's complexity — hundreds of billions of parameters.

3

Autoregressive

It predicts the next token, one token at a time, conditioned on everything written so far.

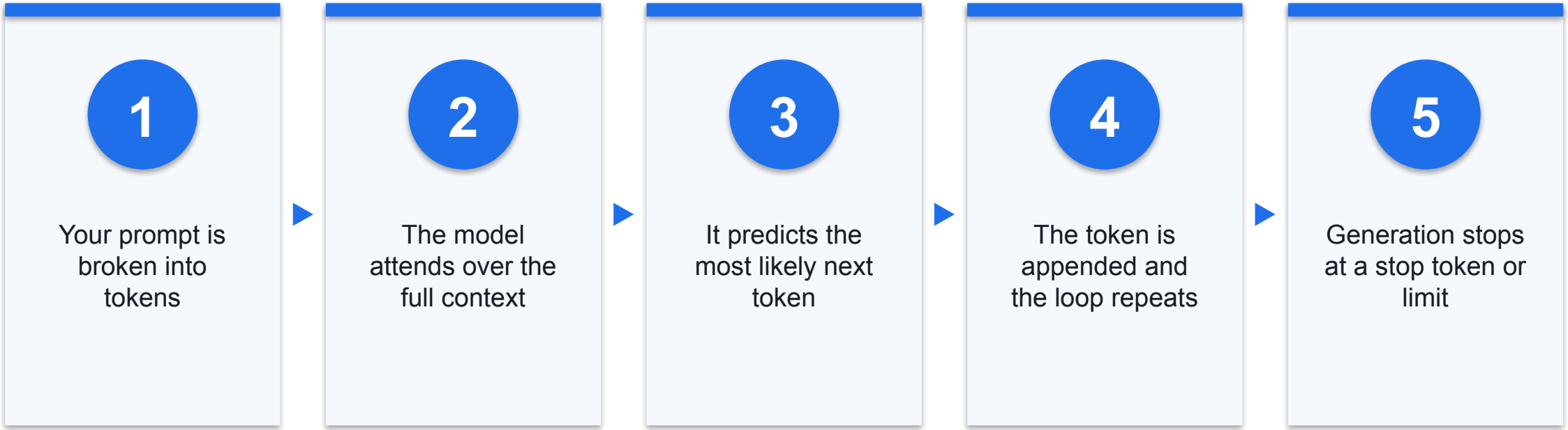
4

Trained then inferred

Training builds the model from a huge corpus; inference is the prompt-and-response you use every day.

AUTOREGRESSIVE GENERATION

How an LLM Produces an Answer



Opportunities of LLM Applications

1. Content creation and writing assistance
2. Customer support and chatbots
3. Language translation and localisation
4. Educational and tutoring tools
5. Business intelligence and analytics
6. Coding and software development
7. Legal and compliance assistance
8. Healthcare and clinical documentation support
9. Enhanced search and knowledge discovery
10. Accessibility for persons with disabilities

Industry Use Cases of Generative AI

Finance & Legal

- Automated financial reporting
- Risk assessment and analysis
- Fraud detection
- Contract review and drafting
- Regulatory compliance monitoring

Marketing & eCommerce

- Content and copy generation
- Personalised recommendations
- Customer sentiment analysis
- SEO and campaign optimisation
- Product description generation

Education & Healthcare

- Personalised learning assistance
- Automated grading and feedback
- Clinical documentation
- Medical imaging support
- Virtual health assistants

THE LANDSCAPE

Popular Large Language Models

1

Google Gemini

Natively multimodal; Ultra, Pro, Flash and Nano variants — the model family used in this course.

2

OpenAI GPT

Widely adopted general-purpose family; reachable from ADK through the LiteLlm wrapper.

3

Anthropic Claude

Strong long-context reasoning and writing; also reachable through LiteLlm.

4

Open models

Meta Llama and Mistral can be self-hosted or run locally through Ollama for data-residency needs.

WHY AGENTS

An LLM answers. An agent acts.

Agentic AI adds reasoning, memory, tool use and autonomy on top of a language model — so it can complete tasks, not just produce text.

THE SIX INGREDIENTS

What Makes a System 'Agentic'?

1

Reasoning

The model plans a sequence of steps instead of answering in one shot.

2

Tool use

It calls functions and APIs to fetch live data or take real actions.

3

Memory

Session state lets it carry context across many turns of a conversation.

4

Autonomy

It decides which tool or which specialist agent handles the request.

5

Collaboration

Multiple specialised agents divide the work and hand off between themselves.

6

Guardrails

Callbacks and policies constrain what the agent is allowed to do.

LLM vs Agent vs Multi-Agent System

Dimension	Plain LLM	Single Agent	Multi-Agent System
What it does	Generates text	Completes a task	Divides work across specialists
Memory	None — stateless	Session state	Shared session + per-agent state
Tools	None	Its own tool set	Each agent has its own tools
Best for	Drafting, summarising	One well-defined job	Broad domains, varied requests
Main risk	Hallucination	Overloaded instruction	Wrong routing / handoff loops

WHEN IT MATTERS Add agents only when one instruction can no longer describe the job clearly — complexity you don't need is complexity you must still debug.

THE PATTERN BEHIND EVERY ADK AGENT

The Agentic Loop — Reason, Act, Observe

WHY IT MATTERS

The loop is what separates an agent from a chatbot: it can take several actions before answering, and it decides how many.

LO3 · WORKED EXAMPLE

Reading an ADK Agent Definition

Every agent in this course is built from these same fields — learn to read them once.

```
from google.adk.agents import Agent

agent = Agent(
    model='gemini-2.0-flash',
    name='banking_assistant',
    description='Handles retail banking queries',
    instruction=(
        'You are a bank assistant. '
        'Never reveal a PIN or OTP.'
    ),
    tools=[get_balance],
)
```

model

Which LLM runs the agent. Flash is fast and cheap; Pro reasons harder.

name

The identifier used in traces and in agent-to-agent transfer.

description

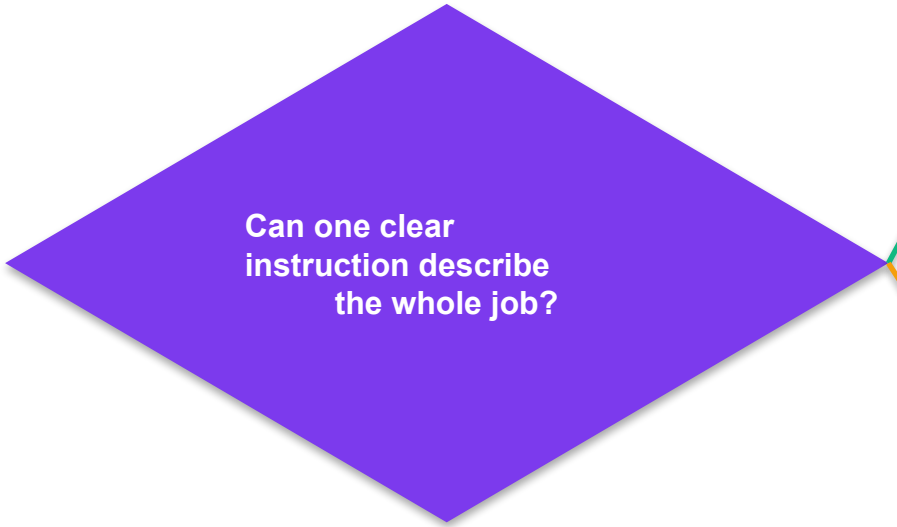
Read by a PARENT agent to decide whether to hand off here.

instruction

The system prompt — role, rules and refusals. Most behaviour lives here.

LO3 · DESIGN DECISION

Which Agent Pattern Should You Use?



YES — use a single agent

Give it the tools it needs. Simpler to build, cheaper to run and far easier to debug. Most business tasks stop here.

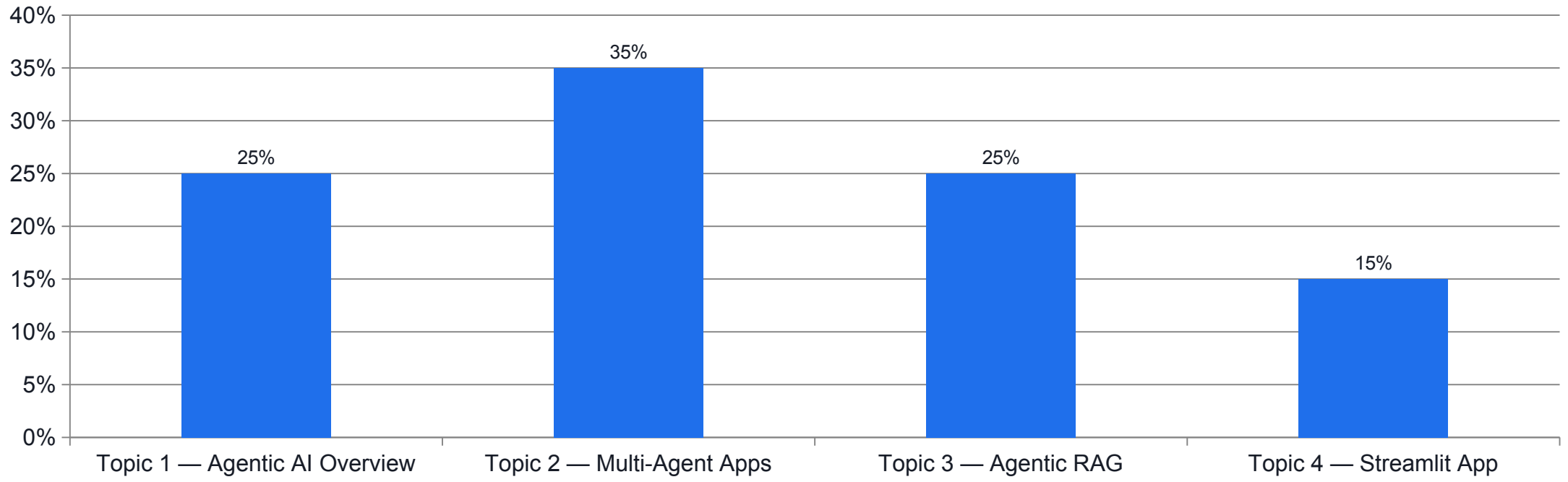
NO — use sub-agents / workflow

Split by specialism. Use sub-agents for routing by intent, or a SequentialAgent when the order of steps is fixed.

A common mistake is starting multi-agent. Start single, and split only when the instruction starts listing unrelated responsibilities.

LESSON PLAN · TIME ALLOCATION

Where Course Time Goes — Topic Weighting

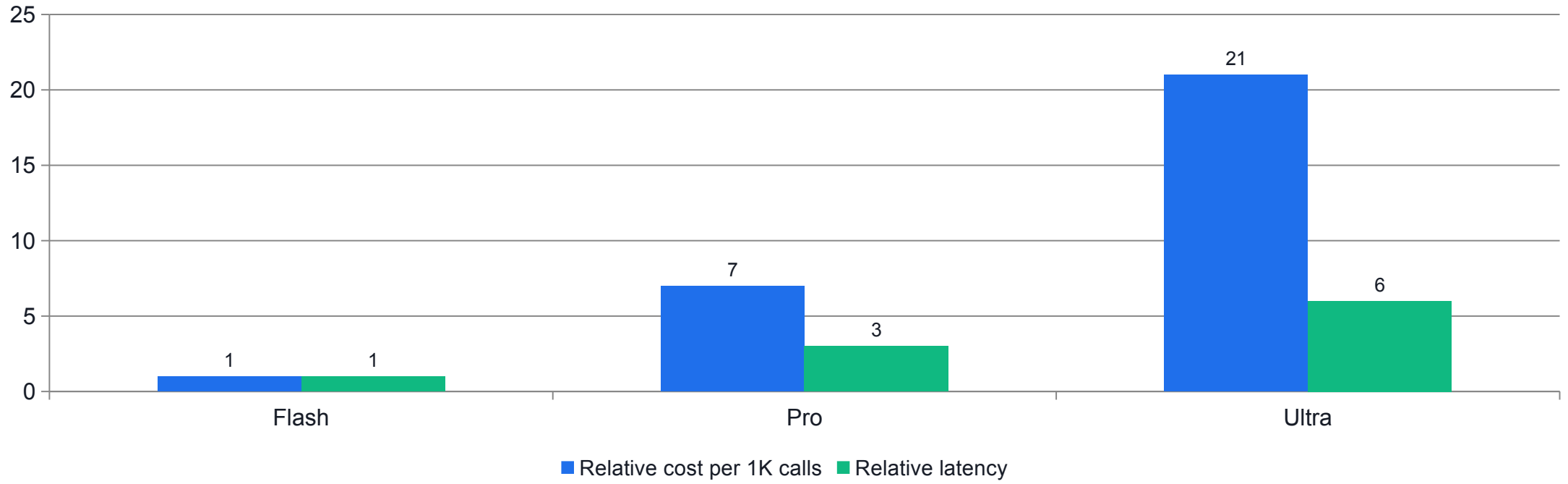


WHAT THE DATA SHOWS

Topic 2 carries the largest weighting because multi-agent orchestration is where most of the assessable practical skill sits — plan your revision accordingly.

LO2 · ENGINEERING TRADE-OFF

Cost and Latency by Model Tier



WHAT THE DATA SHOWS

Moving from Flash to Ultra costs roughly 20x more and runs about 6x slower. Reach for the larger model only where the reasoning genuinely requires it — indicative figures for teaching the trade-off, not a price list.

Gemini Model Selection — Trade-offs

Model	Speed	Cost	Use it when
Gemini Flash	Fastest	Lowest	High-volume agents, tool routing, most labs in this course
Gemini Pro	Moderate	Medium	Complex reasoning, long documents, quality-critical answers
Gemini Ultra	Slowest	Highest	The hardest reasoning tasks where quality outweighs cost
Gemini Nano	On-device	None	Offline or mobile scenarios with privacy constraints

WHEN IT MATTERS Model choice is an engineering trade-off, not a preference — justify it on latency, cost per call and required reasoning depth.

ARCHITECTURE CHOICE

Single Agent vs Multi-Agent

Simple

- Single agent
  - One instruction, one set of tools
  - Simple to build and to debug
  - Instruction grows unwieldy as scope widens
  - Best for a narrow, well-defined job

Scalable

- Multi-agent
  - Each specialist has its own instruction and tools
  - Scales to broad problem domains
  - Routing and handoff must be designed carefully
  - Best for varied requests across several sub-domains

THE MODEL BEHIND ADK

What is Gemini?

1

Built by Google DeepMind

Gemini is Google's flagship model family, built from the ground up for multimodality.

2

Natively multimodal

It reasons across text, images, video, audio and code in a single model.

3

Long context

Very large context windows allow whole documents, codebases and videos in one prompt.

4

Tool-ready

Native function calling makes Gemini a natural fit for agentic applications.

The Gemini Model Family

Pro

- The mid-tier, balanced option
- Strong reasoning, broad multimodal skill
- Optimised for performance, cost and latency
- The default choice for most agents

Flash

- Lightweight and fast
- Optimised for speed and efficiency
- Low cost per call at high volume
- Used in most labs in this course

Ultra & Nano

- Ultra: the most capable, for highly complex tasks
- Nano: the most efficient, runs on-device
- Nano targets mobile and offline scenarios
- Choose by task complexity and where it runs

Model Parameters You Can Tune

1

Temperature

Controls randomness. Low values give focused, repeatable answers; high values give creative, varied ones.

2

Top-K

Restricts sampling to the K most probable next tokens. Top-K of 1 always picks the single most likely token.

3

Top-P

Samples from the smallest set of tokens whose probabilities sum to P — a dynamic alternative to Top-K.

4

Max output tokens

Caps the length of the response, which directly caps cost.

5

Stop sequences

Strings that end generation immediately when produced — useful for structured formats.

6

Safety settings

Thresholds that filter harmful content across several harm categories.

TOPIC 1 PREVIEW

What is the Agent Development Kit (ADK)?

1

Open-source framework

Google's Python framework for building, evaluating and deploying AI agents.

2

Model-agnostic

Runs on Gemini natively, and on OpenAI, Anthropic or local models through LiteLlm.

3

Batteries included

Sessions, tools, callbacks, multi-agent orchestration and evaluation are all built in.

4

Developer UI

adk web gives you a local browser IDE that shows every event, tool call and trace.

THE SIX FIELDS

Anatomy of an ADK Agent

1

model

Which LLM powers the agent — for example gemini-2.0-flash.

2

name

A unique identifier used in traces and in agent-to-agent transfers.

3

description

What this agent is for. A parent agent reads this to decide whether to hand off to it.

4

instruction

The system prompt: the agent's role, rules, tone and boundaries.

5

tools

Python functions the agent may call to fetch data or take action.

6

sub_agents

Specialist agents this agent may transfer control to.

HOW A TURN EXECUTES

The ADK Agent Loop

1. User message enters the Runner
2. Session history is loaded as context
3. The model reasons and may emit a tool call
4. The tool runs and its result returns to the model
5. The final response is streamed back and stored

01

TOPIC 01

Overview of Agentic AI in Gemini ADK

LLM & agentic AI foundations · Gemini model family · ADK architecture · first agent · tools

TOPIC WEIGHTING 25%

Key Concepts — Overview of Agentic AI in Gemini ADK

1. A Large Language Model (LLM) is trained on very large text corpora and generates language autoregressively, one token at a time.

2. Agentic AI adds reasoning, memory, tool use and autonomy on top of an LLM, so the model can act, not just answer.

3. Gemini is Google DeepMind's natively multimodal model family — Ultra, Pro, Flash and Nano — reasoning across text, image, video, audio and code.

4. The Agent Development Kit (ADK) is Google's open-source Python framework for building, evaluating and deploying agents.

TOPIC WEIGHTING 25%

Key Concepts — Overview of Agentic AI in Gemini ADK (continued)

1. An ADK Agent is defined by four things: a model, a name, a description and an instruction.

2. Tools are ordinary Python functions the agent may call; the docstring and type hints tell the model when and how to call them.

3. adk web launches a local browser IDE for chatting with an agent and inspecting every event, tool call and trace.

4. LiteLlm lets an ADK agent run on non-Gemini models (OpenAI, Anthropic, Ollama) without changing agent code.

WHAT YOU'LL DO

Hands-On Labs — Overview of Agentic AI in Gemini ADK

Labs 1–2

- Set Up the Gemini ADK Environment and Get an API Key
- Build Your First ADK Agent — A Retail Banking Assistant

Labs 3–4

- Give an Agent Tools — Live Weather and Web Search
- Swap the Model — Running an ADK Agent on a Non-Gemini LLM

—

- —

TOPIC 01 · HANDS-ON

Set Up the Gemini ADK Environment and Get an API Key

LAB 1

LO1 / LO2 — establish a working Gemini ADK development environment.

Install the Agent Development Kit toolchain with uv, obtain a free Google AI Studio API key, and store it safely in a .env file so every agent in the course can authenticate.

YOU'LL BUILD

A working Python 3.13 project with google-adk installed and a validated GOOGLE_API_KEY.

TOOLCHAIN

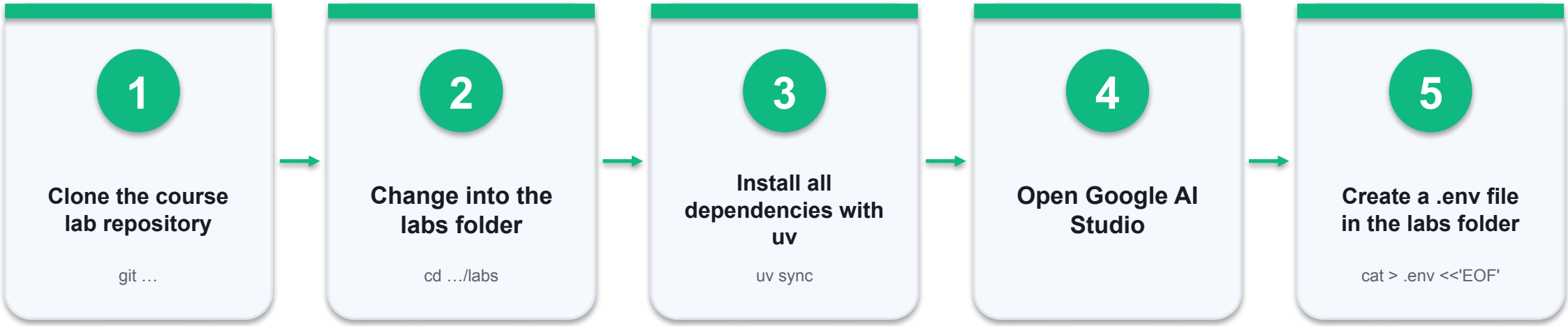
Google AI Studio, uv, Python 3.13, google-adk, python-dotenv

DONE WHEN

uv run adk --help prints the ADK usage banner listing the run, web and eval sub-commands, and no ModuleNotFoundError is raised.

LAB 1 · PROCESS MAP

How Lab 1 Runs



THE POINT

LO1 / LO2 — establish a working Gemini ADK development environment.

LAB 1 · PROCEDURE

Set Up the Gemini ADK Environment and Get an API Key — Part 1/2

1

Clone the course lab repository

```
$ git clone https://github.com/tertiarycourses/TGS-2024042961-Develop-Multi-AI-Agent-Applicatio...
```

2

Change into the labs folder

```
$ cd TGS-2024042961-Develop-Multi-AI-Agent-Applications-with-Gemini-Agent-ADK/labs
```

3

Install all dependencies with uv (creates the .venv automatically)

```
$ uv sync
```

4

Open Google AI Studio and sign in with your Google account, then click Get API key → Create API key

LAB 1 · PROCEDURE

Set Up the Gemini ADK Environment and Get an API Key — Part 2/2

5 Create a .env file in the labs folder and paste your key

```
$ cat > .env <<'EOF'
```

6 Confirm the ADK command-line tool is on the path

```
$ uv run adk --help
```

LAB 1 · VERIFY

Set Up the Gemini ADK Environment and Get an API Key

✅ Expected result

uv run adk --help prints the ADK usage banner listing the run, web and eval sub-commands, and no ModuleNotFoundError is raised.

IF IT DOESN'T WORK

Nothing happens

Check the .env file is in the labs folder and the key has no quotes or spaces.

Auth or 401 error

Re-copy the API key from AI Studio; confirm GOOGLE_GENAI_USE_VERTEXAI=0.

ModuleNotFoundError

Run uv sync again, and prefix commands with uv run so the venv is used.

TOPIC 01 · HANDS-ON

Build Your First ADK Agent — A Retail Banking Assistant

LAB 2

LO1 / LO3 — define an agent from a model, name, description and instruction.

Create a single-agent banking customer-service assistant. You learn the four fields that define every ADK agent and see how the instruction alone shapes tone, scope and refusals.

YOU'LL BUILD

lab02 — a Gemini-powered banking assistant that answers general banking questions and refuses to handle PINs, OTPs or full account numbers.

TOOLCHAIN

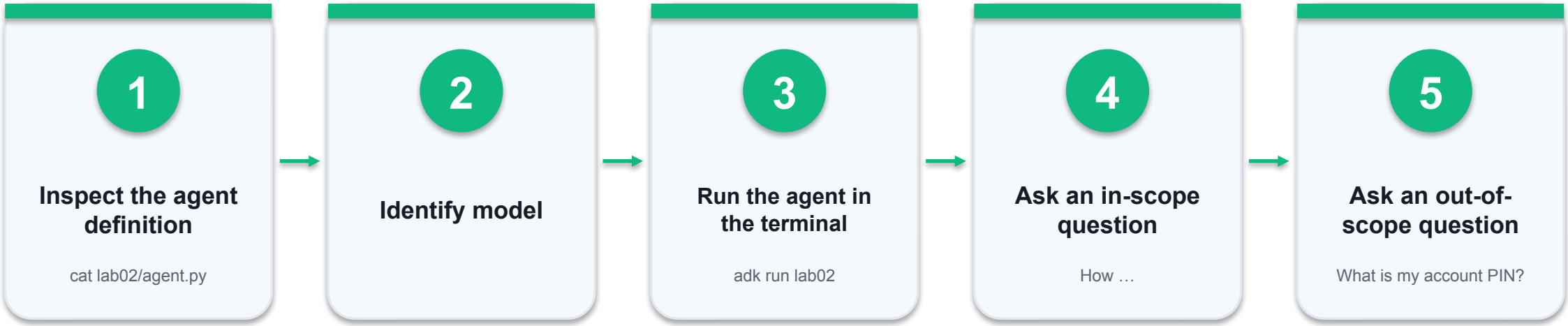
google-adk, Gemini 2.0 Flash, adk run, adk web

DONE WHEN

The agent answers general banking questions helpfully but declines to disclose or request a PIN, OTP or full account number, and the Events tab shows one LLM call per turn.

LAB 2 · PROCESS MAP

How Lab 2 Runs



THE POINT

LO1 / LO3 — define an agent from a model, name, description and instruction.

LAB 2 · PROCEDURE

Build Your First ADK Agent — A Retail Banking Assistant — Part 1/2

1

Inspect the agent definition and note the four required fields

```
$ cat lab02/agent.py
```

2

Identify model, name, description and instruction in the Agent(...) call

3

Run the agent in the terminal

```
$ uv run adk run lab02
```

4

Ask an in-scope question

```
$ How do I reset my internet banking password?
```

LAB 2 · PROCEDURE

Build Your First ADK Agent — A Retail Banking Assistant — Part 2/2

5 **Ask an out-of-scope question and observe the guarded refusal**

```
$ What is my account PIN?
```

6 **Launch the browser IDE and re-run the same prompts**

```
$ uv run adk web
```

7 Open http://localhost:8000, select lab02, and inspect the Events tab

8 Edit the instruction to make the assistant reply only in formal English, then re-run

LAB 2 · VERIFY

Build Your First ADK Agent — A Retail Banking Assistant

✅ Expected result

The agent answers general banking questions helpfully but declines to disclose or request a PIN, OTP or full account number, and the Events tab shows one LLM call per turn.

IF IT DOESN'T WORK

Nothing happens

Check the .env file is in the labs folder and the key has no quotes or spaces.

Auth or 401 error

Re-copy the API key from AI Studio; confirm GOOGLE_GENAI_USE_VERTEXAI=0.

ModuleNotFoundError

Run uv sync again, and prefix commands with uv run so the venv is used.

TOPIC 01 · HANDS-ON

Give an Agent Tools — Live Weather and Web Search

LAB 3

LO1 / LO3 — extend an agent with custom function tools and evaluate tool selection.

Add two Python function tools to an agent: a live OpenWeather lookup and a Tavily web search. You see how the docstring and type hints become the tool contract the model reads.

YOU'LL BUILD

lab03 — an agent that decides for itself whether a question needs the weather tool, the search tool, both, or neither.

TOOLCHAIN

google-adk, OpenWeather API, Tavily API, Gemini 2.0 Flash

DONE WHEN

The weather question produces a get_weather function_call with a live temperature; the news question produces a tavily_search call; the arithmetic question produces no tool call at all.

LAB 3 · PROCESS MAP

How Lab 3 Runs

1. **Register for a free OpenWeather API key at…**
2. **Add both keys to labs/.env as…**
3. **Read the two tool functions**

   cat lab03/agent.py
4. **Observe that tools are attached with…**
5. **Run the agent**

   adk run lab03

THE POINT

LO1 / LO3 — extend an agent with custom function tools and evaluate tool selection.

LAB 3 · PROCEDURE

Give an Agent Tools — Live Weather and Web Search — Part 1/3

1. Register for a free OpenWeather API key at openweathermap.org and a Tavily key at tavily.com

2. Add both keys to labs/.env as OPENWEATHER_API_KEY and TAVILY_API_KEY

3. **Read the two tool functions and note the docstring, the typed arguments and the dict return**

```
$ cat lab03/agent.py
```

4. Observe that tools are attached with tools=[get_weather, tavily_search]

LAB 3 · PROCEDURE

Give an Agent Tools — Live Weather and Web Search — Part 2/3

5

Run the agent

```
$ uv run adk run lab03
```

6

Trigger the weather tool

```
$ What is the weather in Singapore right now?
```

7

Trigger the search tool

```
$ What are the latest announcements about Google Gemini?
```

8

Ask a question needing no tool and confirm none is called

```
$ What is 15 multiplied by 12?
```

LAB 3 · PROCEDURE

Give an Agent Tools — Live Weather and Web Search — Part 3/3

9

Open adk web and read the function_call and function_response events for each turn

```
$ uv run adk web
```

LAB 3 · VERIFY

Give an Agent Tools — Live Weather and Web Search

✅ Expected result

The weather question produces a get_weather function_call with a live temperature; the news question produces a tavily_search call; the arithmetic question produces no tool call at all.

IF IT DOESN'T WORK

Nothing happens

Check the .env file is in the labs folder and the key has no quotes or spaces.

Auth or 401 error

Re-copy the API key from AI Studio; confirm GOOGLE_GENAI_USE_VERTEXAI=0.

ModuleNotFoundError

Run uv sync again, and prefix commands with uv run so the venv is used.

TOPIC 01 · HANDS-ON

Swap the Model — Running an ADK Agent on a Non-Gemini LLM

LAB 4

LO2 / LO4 — compare models and assess the trade-offs for an engineering process.

Use the LiteLlm wrapper to point the same agent at an OpenAI model, then compare it with Gemini on the same prompts to judge quality, latency and cost.

YOU'LL BUILD

lab04 — one agent definition that runs on either Gemini or an OpenAI model by changing a single line.

TOOLCHAIN

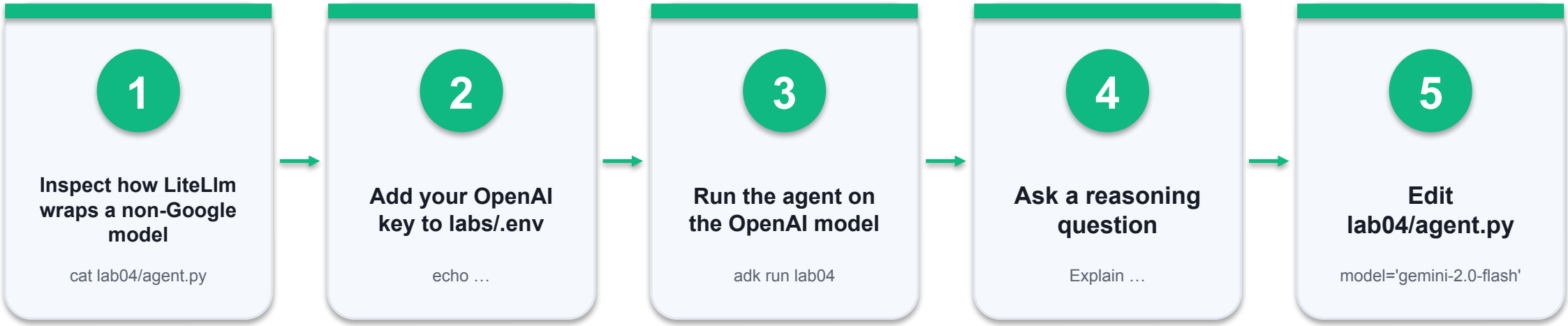
google-adk, LiteLlm, Gemini 2.0 Flash, OpenAI GPT-4.1-mini

DONE WHEN

The same agent runs unchanged on both providers, and you can state which model you would choose for this workload and justify it on quality, latency and cost.

LAB 4 · PROCESS MAP

How Lab 4 Runs



THE POINT

LO2 / LO4 — compare models and assess the trade-offs for an engineering process.

LAB 4 · PROCEDURE

Swap the Model — Running an ADK Agent on a Non-Gemini LLM — Part 1/2

1

Inspect how LiteLlm wraps a non-Google model

```
$ cat lab04/agent.py
```

2

Add your OpenAI key to labs/.env

```
$ echo 'OPENAI_API_KEY=your-openai-key' >> .env
```

3

Run the agent on the OpenAI model

```
$ uv run adk run lab04
```

4

Ask a reasoning question and note the answer quality and response time

```
$ Explain in three sentences why an agent needs tools.
```

LAB 4 · PROCEDURE

Swap the Model — Running an ADK Agent on a Non-Gemini LLM — Part 2/2

5 **Edit lab04/agent.py and replace the model with a Gemini model string**

```
$ model='gemini-2.0-flash'
```

6 **Re-run the identical prompt on Gemini**

```
$ uv run adk run lab04
```

7 Record quality, latency and cost for both in a comparison table

LAB 4 · VERIFY

Swap the Model — Running an ADK Agent on a Non-Gemini LLM

✅ Expected result

The same agent runs unchanged on both providers, and you can state which model you would choose for this workload and justify it on quality, latency and cost.

IF IT DOESN'T WORK

Nothing happens

Check the .env file is in the labs folder and the key has no quotes or spaces.

Auth or 401 error

Re-copy the API key from AI Studio; confirm GOOGLE_GENAI_USE_VERTEXAI=0.

ModuleNotFoundError

Run uv sync again, and prefix commands with uv run so the venv is used.

TOPIC RECAP

Recap — Overview of Agentic AI in Gemini ADK

- You can now: LO1 / LO2 — establish a working Gemini ADK development environment.
- You can now: LO1 / LO3 — define an agent from a model, name, description and instruction.
- You can now: LO1 / LO3 — extend an agent with custom function tools and evaluate tool selection.
- You can now: LO2 / LO4 — compare models and assess the trade-offs for an engineering process.

02

TOPIC 02

Build A Multi Agent App with Gemini ADK

Sessions & state · handoff · sub-agents · workflow agents · guardrails · structured output · MCP

TOPIC WEIGHTING 35%

Key Concepts — Build A Multi Agent App with Gemini ADK

1. LLMs are stateless — a Session plus a SessionService is what gives an ADK agent memory across turns.

2. The Runner is the execution engine: it takes a user message, drives the agent loop and streams back Events.

3. Multi-agent handoff — a root agent with sub_agents transfers control to whichever specialist matches the request.

4. A hierarchical (coordinator) pattern puts a router agent above specialised sub-agents, each with its own instruction and tools.

5. SequentialAgent runs sub-agents in a fixed order and passes each output forward; ParallelAgent fans them out concurrently.

TOPIC WEIGHTING 35%

Key Concepts — Build A Multi Agent App with Gemini ADK (continued)

1. Callbacks such as before_model_callback implement guardrails — inspect, block or rewrite a request before it reaches the model.

2. output_schema with a Pydantic model forces the agent to emit validated, machine-readable JSON instead of free text.

3. The Model Context Protocol (MCP) is an open standard that lets an agent consume tools hosted by an external server over StreamableHTTP or SSE.

4. Agents can also be declared declaratively in YAML config files instead of Python.

WHAT YOU'LL DO

Hands-On Labs — Build A Multi Agent App with Gemini ADK

Labs 5–7

- Give an Agent Memory — Sessions, State and the Runner
- Inspect the Agent Loop — Events, Tool Calls and Final Responses
- Multi-Agent Handoff — Joke Generator to Translator

Labs 8–10

- Hierarchical Multi-Agent System — The Tutor Agent
- Sequential Workflow Agent — Singapore Transport Route Planner
- Add a Guardrail — Blocking Unsafe Requests with a Callback

Labs 11–12

- Structured Output — Forcing Valid JSON with Pydantic
- Connect External Tools with MCP — StreamableHTTP and SSE

TOPIC 02 · HANDS-ON

Give an Agent Memory — Sessions, State and the Runner

LAB 5

LO3 — implement session management so an agent remembers earlier turns.

Drive an agent programmatically with a Runner and an InMemorySessionService, so the conversation persists across turns instead of restarting on every message.

YOU'LL BUILD

lab05 — a multi-turn agent that answers follow-up questions using earlier context.

TOOLCHAIN

google-adk, Runner, InMemorySessionService, google.genai types

DONE WHEN

The follow-up 'And what about Osaka?' is understood as a weather question without repeating the word weather — and stops working when the session is removed.

LAB 5 · PROCESS MAP

How Lab 5 Runs

1. **Read how the session service**
   cat lab05/agent.py
2. **Note the APP_NAME**
3. **Run the session script**
   lab05/agent.py
4. **Ask an initial question that establishes…**
   What is the weather in…
5. **Ask a follow-up that only works if the…**
   And what about Osaka?

THE POINT

LO3 — implement session management so an agent remembers earlier turns.

LAB 5 · PROCEDURE

Give an Agent Memory — Sessions, State and the Runner — Part 1/2

1

Read how the session service, session and Runner are wired together

```
$ cat lab05/agent.py
```

2

Note the APP_NAME, USER_ID and SESSION_ID that identify one conversation

3

Run the session script

```
$ uv run python lab05/agent.py
```

4

Ask an initial question that establishes context

```
$ What is the weather in Tokyo?
```

LAB 5 · PROCEDURE

Give an Agent Memory — Sessions, State and the Runner — Part 2/2

5 **Ask a follow-up that only works if the agent remembers**

```
$ And what about Osaka?
```

6 Comment out the session creation and re-run to observe the failure

7 Restore the session code and confirm continuity is back

LAB 5 · VERIFY

Give an Agent Memory — Sessions, State and the Runner

✅ Expected result

The follow-up 'And what about Osaka?' is understood as a weather question without repeating the word weather — and stops working when the session is removed.

IF IT DOESN'T WORK

Nothing happens

Check the .env file is in the labs folder and the key has no quotes or spaces.

Auth or 401 error

Re-copy the API key from AI Studio; confirm GOOGLE_GENAI_USE_VERTEXAI=0.

ModuleNotFoundError

Run uv sync again, and prefix commands with uv run so the venv is used.

TOPIC 02 · HANDS-ON

Inspect the Agent Loop — Events, Tool Calls and Final Responses

LAB 6

LO3 / LO4 — analyse the agent execution loop and evaluate its behaviour.

Stream the Event objects the Runner emits and learn to read an agent trace: which event carries the tool call, which carries the tool result, and which is the final response.

YOU'LL BUILD

lab06 — an instrumented agent that prints every event in its reasoning loop.

TOOLCHAIN

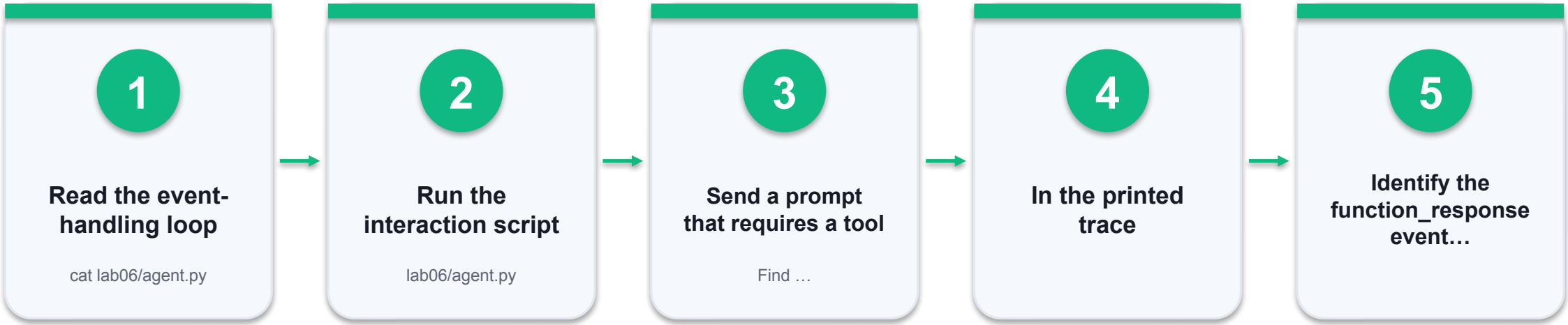
google-adk, Runner, Events API, InMemorySessionService

DONE WHEN

You can point to the function_call, the function_response and the final response in the trace, and state the number of LLM round-trips the turn consumed.

LAB 6 · PROCESS MAP

How Lab 6 Runs



THE POINT

LO3 / LO4 — analyse the agent execution loop and evaluate its behaviour.

LAB 6 · PROCEDURE

Inspect the Agent Loop — Events, Tool Calls and Final Responses — Part 1/2

1

Read the event-handling loop and find is_final_response()

```
$ cat lab06/agent.py
```

2

Run the interaction script

```
$ uv run python lab06/agent.py
```

3

Send a prompt that requires a tool

```
$ Find the weather in Singapore and summarise recent AI news.
```

4

In the printed trace, identify the function_call event

LAB 6 · PROCEDURE

Inspect the Agent Loop — Events, Tool Calls and Final Responses — Part 2/2

5 Identify the function_response event carrying the tool's return value

6 Identify the final response event and note how many LLM calls one turn actually took

7 Explain why a two-tool question produces more events than a one-tool question

LAB 6 · VERIFY

Inspect the Agent Loop — Events, Tool Calls and Final Responses

✅ Expected result

You can point to the function_call, the function_response and the final response in the trace, and state the number of LLM round-trips the turn consumed.

IF IT DOESN'T WORK

Nothing happens

Check the .env file is in the labs folder and the key has no quotes or spaces.

Auth or 401 error

Re-copy the API key from AI Studio; confirm GOOGLE_GENAI_USE_VERTEXAI=0.

ModuleNotFoundError

Run uv sync again, and prefix commands with uv run so the venv is used.

TOPIC 02 · HANDS-ON

Multi-Agent Handoff — Joke Generator to Translator

LAB 7

LO3 — implement agent-to-agent delegation with sub_agents.

Build a three-level agent hierarchy where a root agent hands off to a joke generator, which in turn hands off to a translator. This is the core ADK delegation pattern.

YOU'LL BUILD

lab07 — a root agent that produces an English joke and its Chinese translation through two automatic handoffs.

TOOLCHAIN

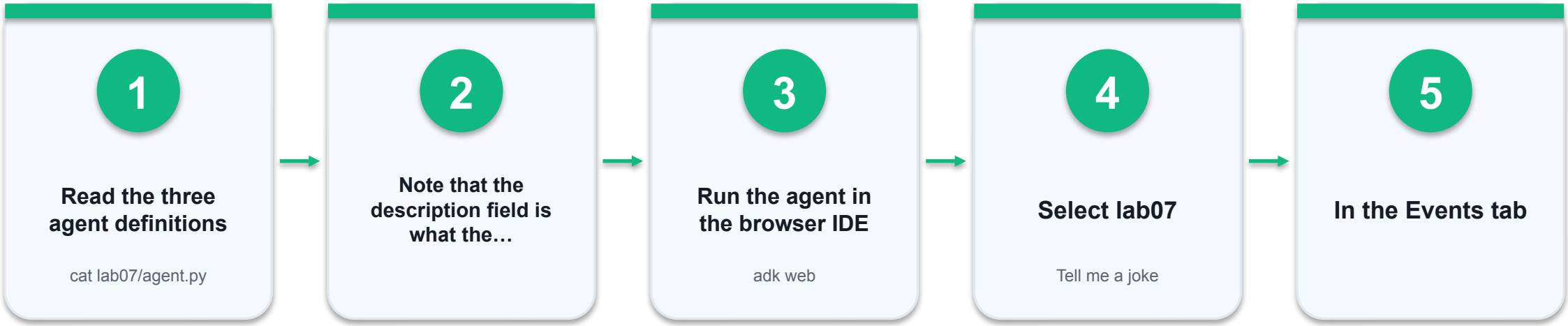
google-adk, sub_agents, Gemini 2.0 Flash

DONE WHEN

One 'Tell me a joke' request yields an English joke followed by a Chinese translation, and the Events tab shows two transfer_to_agent calls.

LAB 7 · PROCESS MAP

How Lab 7 Runs



THE POINT

LO3 — implement agent-to-agent delegation with sub_agents.

LAB 7 · PROCEDURE

Multi-Agent Handoff — Joke Generator to Translator — Part 1/2

1

Read the three agent definitions and the sub_agents chain

```
$ cat lab07/agent.py
```

2

Note that the description field is what the parent reads to decide on a handoff

3

Run the agent in the browser IDE

```
$ uv run adk web
```

4

Select lab07 and request a joke

```
$ Tell me a joke
```

LAB 7 · PROCEDURE

Multi-Agent Handoff — Joke Generator to Translator — Part 2/2

5 In the Events tab, find the transfer_to_agent call into joke_generator

6 Find the second transfer into translator and confirm the Chinese output

7 Weaken the translator's description to one vague word and re-run

8 Observe the handoff becoming unreliable, then restore the description

LAB 7 · VERIFY

Multi-Agent Handoff — Joke Generator to Translator

✅ Expected result

One 'Tell me a joke' request yields an English joke followed by a Chinese translation, and the Events tab shows two transfer_to_agent calls.

IF IT DOESN'T WORK

Nothing happens

Check the .env file is in the labs folder and the key has no quotes or spaces.

Auth or 401 error

Re-copy the API key from AI Studio; confirm GOOGLE_GENAI_USE_VERTEXAI=0.

ModuleNotFoundError

Run uv sync again, and prefix commands with uv run so the venv is used.

TOPIC 02 · HANDS-ON

Hierarchical Multi-Agent System — The Tutor Agent

LAB 8

LO3 — design a coordinator agent routing to specialised sub-agents.

Build a tutoring system where one root agent routes each question to a maths, physics or history specialist, each with its own instruction and teaching style.

YOU'LL BUILD

lab08 — a coordinator with three subject specialists that routes by topic.

TOOLCHAIN

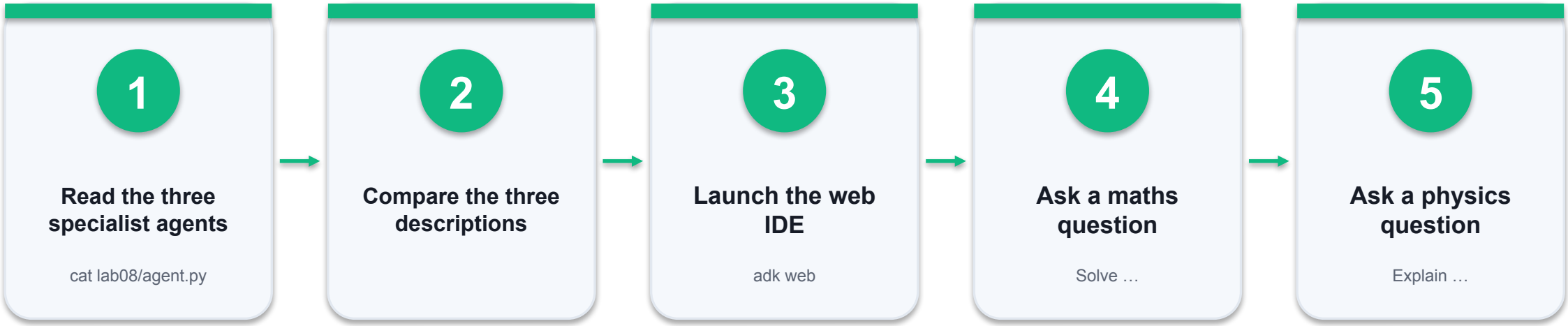
google-adk, sub_agents, coordinator pattern

DONE WHEN

Each subject question is answered by the matching specialist, visible as a transfer_to_agent event, and your new fourth specialist is routed to correctly.

LAB 8 · PROCESS MAP

How Lab 8 Runs



THE POINT

LO3 — design a coordinator agent routing to specialised sub-agents.

LAB 8 · PROCEDURE

Hierarchical Multi-Agent System — The Tutor Agent — Part 1/2

1. **Read the three specialist agents and the root coordinator**

```
$ cat lab08/agent.py
```

2. Compare the three descriptions and note how each states its routing trigger

3. **Launch the web IDE and select lab08**

```
$ uv run adk web
```

4. **Ask a maths question and confirm it routes to math_tutor_agent**

```
$ Solve 2x + 5 = 17 step by step.
```

LAB 8 · PROCEDURE

Hierarchical Multi-Agent System — The Tutor Agent — Part 2/2

5 **Ask a physics question**

```
$ Explain Newton's second law with an example.
```

6 **Ask a history question**

```
$ What caused the fall of the Roman Empire?
```

7 **Ask an ambiguous cross-subject question and observe how the router resolves it**

```
$ How did physics change during the Industrial Revolution?
```

8 Add a fourth specialist of your own choosing to sub_agents and test the routing

LAB 8 · VERIFY

Hierarchical Multi-Agent System — The Tutor Agent

✅ Expected result

Each subject question is answered by the matching specialist, visible as a transfer_to_agent event, and your new fourth specialist is routed to correctly.

IF IT DOESN'T WORK

Nothing happens

Check the .env file is in the labs folder and the key has no quotes or spaces.

Auth or 401 error

Re-copy the API key from AI Studio; confirm GOOGLE_GENAI_USE_VERTEXAI=0.

ModuleNotFoundError

Run uv sync again, and prefix commands with uv run so the venv is used.

TOPIC 02 · HANDS-ON

Sequential Workflow Agent — Singapore Transport Route Planner

LAB 9

LO3 / LO2 — orchestrate a fixed multi-stage pipeline with SequentialAgent.

Chain three agents in a fixed order — collect the journey, research cross-country options, then produce a full route report — using SequentialAgent with the google_search tool.

YOU'LL BUILD

lab09 — a sequential pipeline producing a route report by MRT, bus, taxi, cycling and walking.

TOOLCHAIN

google-adk, SequentialAgent, LlmAgent, google_search

DONE WHEN

A single journey request produces one report containing all five transport modes plus a recommended fastest route, with the three stages visible in order in the Events tab.

LAB 9 · PROCESS MAP

How Lab 9 Runs

1. **Read the three sub-agents**
   cat lab09/agent.py
2. **Note that each agent's output is passed…**
3. **Launch the web IDE**
   adk web
4. **Provide a journey when the first agent asks**
   From …
5. **Wait for all three stages to complete**

THE POINT

LO3 / LO2 — orchestrate a fixed multi-stage pipeline with SequentialAgent.

LAB 9 · PROCEDURE

Sequential Workflow Agent — Singapore Transport Route Planner — Part 1/2

1

Read the three sub-agents and the SequentialAgent that orders them

```
$ cat lab09/agent.py
```

2

Note that each agent's output is passed forward as the next agent's input

3

Launch the web IDE and select lab09

```
$ uv run adk web
```

4

Provide a journey when the first agent asks

```
$ From Jurong East to Changi Airport
```

LAB 9 · PROCEDURE

Sequential Workflow Agent — Singapore Transport Route Planner — Part 2/2

5 Wait for all three stages to complete and read the consolidated report

6 Verify the report covers bus, MRT, taxi, cycling, walking and a fastest route

7 Explain why SequentialAgent, not sub_agents handoff, is the right pattern here

LAB 9 · VERIFY

Sequential Workflow Agent — Singapore Transport Route Planner

✅ Expected result

A single journey request produces one report containing all five transport modes plus a recommended fastest route, with the three stages visible in order in the Events tab.

IF IT DOESN'T WORK

Nothing happens

Check the .env file is in the labs folder and the key has no quotes or spaces.

Auth or 401 error

Re-copy the API key from AI Studio; confirm GOOGLE_GENAI_USE_VERTEXAI=0.

ModuleNotFoundError

Run uv sync again, and prefix commands with uv run so the venv is used.

TOPIC 02 · HANDS-ON

Add a Guardrail — Blocking Unsafe Requests with a Callback

LAB 10

LO3 / LO4 — implement and evaluate a safety guardrail on an agent.

Use before_model_callback to inspect every user message before it reaches the LLM, block requests containing a forbidden keyword, and record the block in session state.

YOU'LL BUILD

lab10 — a tool-using agent that intercepts and refuses blocked requests without ever calling the model.

TOOLCHAIN

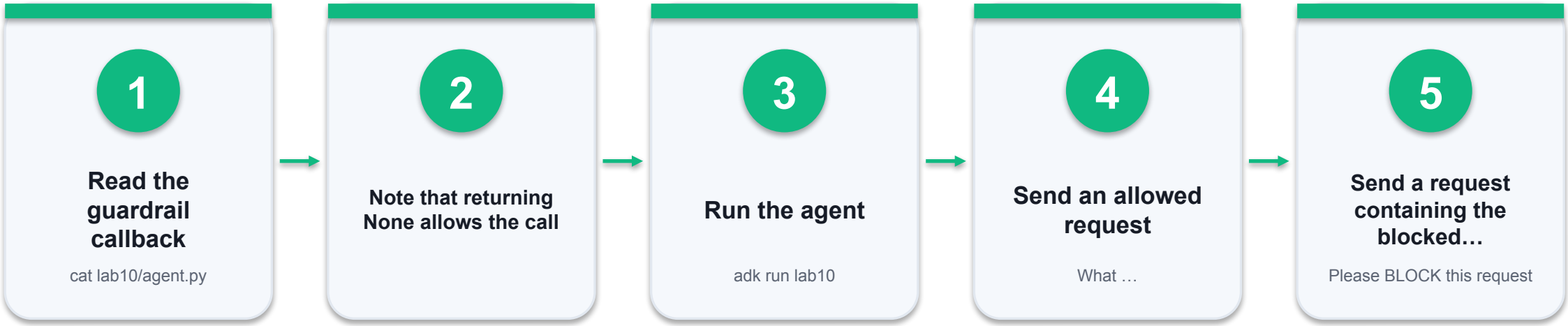
google-adk, before_model_callback, CallbackContext, LlmRequest, LlmResponse

DONE WHEN

The blocked keyword returns the refusal message with no model call in the trace, while normal requests still work — and your added keyword is blocked too.

LAB 10 · PROCESS MAP

How Lab 10 Runs



THE POINT

LO3 / LO4 — implement and evaluate a safety guardrail on an agent.

LAB 10 · PROCEDURE

Add a Guardrail — Blocking Unsafe Requests with a Callback — Part 1/2

1. **Read the guardrail callback and find where it returns an LlmResponse**

```
$ cat lab10/agent.py
```

2. Note that returning None allows the call and returning a response blocks it

3. **Run the agent**

```
$ uv run adk run lab10
```

4. **Send an allowed request and confirm it reaches the model**

```
$ What is the weather in Singapore?
```

LAB 10 · PROCEDURE

Add a Guardrail — Blocking Unsafe Requests with a Callback — Part 2/2

5 **Send a request containing the blocked keyword**

```
$ Please BLOCK this request
```

6 Confirm the refusal message is returned and no LLM call was made

7 Extend the guardrail to also block a second keyword of your choice

8 Re-run and verify both keywords are now intercepted

LAB 10 · VERIFY

Add a Guardrail — Blocking Unsafe Requests with a Callback

✅ Expected result

The blocked keyword returns the refusal message with no model call in the trace, while normal requests still work — and your added keyword is blocked too.

IF IT DOESN'T WORK

Nothing happens

Check the .env file is in the labs folder and the key has no quotes or spaces.

Auth or 401 error

Re-copy the API key from AI Studio; confirm GOOGLE_GENAI_USE_VERTEXAI=0.

ModuleNotFoundError

Run uv sync again, and prefix commands with uv run so the venv is used.

TOPIC 02 · HANDS-ON

Structured Output — Forcing Valid JSON with Pydantic

LAB 11

LO3 — produce machine-readable agent output for downstream systems.

Attach a Pydantic output_schema so the agent returns a validated Recipe object with typed fields instead of free-form prose that a downstream system cannot parse.

YOU'LL BUILD

lab11 — an agent whose every reply is schema-valid JSON.

TOOLCHAIN

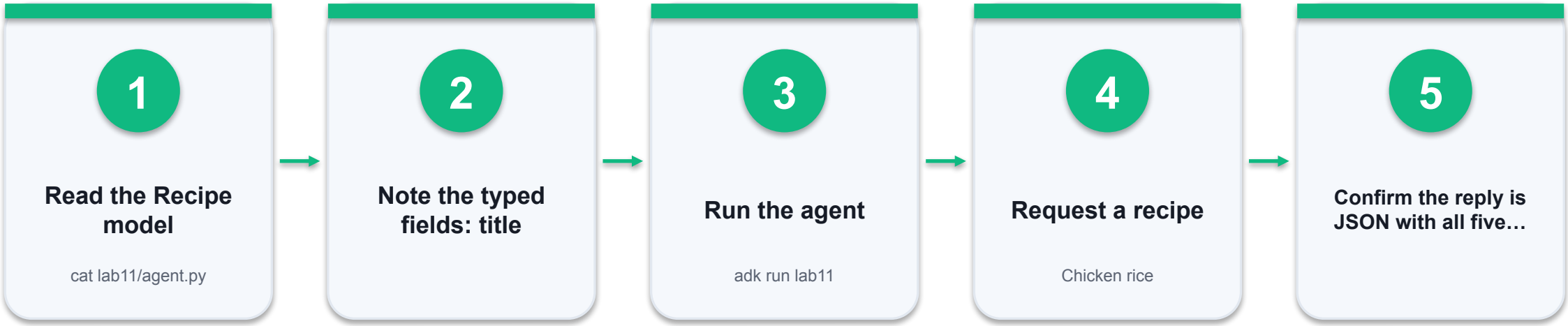
google-adk, Pydantic BaseModel, output_schema

DONE WHEN

Every response parses as JSON matching the Recipe schema, cooking_time is an integer, and your added difficulty field is populated.

LAB 11 · PROCESS MAP

How Lab 11 Runs



THE POINT

LO3 — produce machine-readable agent output for downstream systems.

LAB 11 · PROCEDURE

Structured Output — Forcing Valid JSON with Pydantic — Part 1/2

1

Read the Recipe model and the output_schema argument

```
$ cat lab11/agent.py
```

2

Note the typed fields: title, ingredients, cooking_time, servings, instructions

3

Run the agent

```
$ uv run adk run lab11
```

4

Request a recipe

```
$ Chicken rice
```

LAB 11 · PROCEDURE

Structured Output — Forcing Valid JSON with Pydantic — Part 2/2

5 Confirm the reply is JSON with all five fields and correct types

6 **Add a difficulty field to the Recipe model**

```
$ difficulty: str
```

7 Re-run and confirm the new field appears in the output

8 Note the ADK restriction that an agent with output_schema cannot also use tools

LAB 11 · VERIFY

Structured Output — Forcing Valid JSON with Pydantic

✅ Expected result

Every response parses as JSON matching the Recipe schema, cooking_time is an integer, and your added difficulty field is populated.

IF IT DOESN'T WORK

Nothing happens

Check the .env file is in the labs folder and the key has no quotes or spaces.

Auth or 401 error

Re-copy the API key from AI Studio; confirm GOOGLE_GENAI_USE_VERTEXAI=0.

ModuleNotFoundError

Run uv sync again, and prefix commands with uv run so the venv is used.

TOPIC 02 · HANDS-ON

Connect External Tools with MCP — StreamableHTTP and SSE

LAB 12

LO3 / LO2 — integrate third-party tools through the Model Context Protocol.

Connect an ADK agent to a remote MCP server so it can use tools it does not own, using both the StreamableHTTP and SSE transports.

YOU'LL BUILD

lab12 — agent.py (StreamableHTTP) and agent_sse.py (SSE), whose toolset is discovered at runtime from an MCP server.

TOOLCHAIN

google-adk, McpToolset, StreamableHTTPConnectionParams, SseConnectionParams, n8n MCP server

DONE WHEN

The agent prints the tools discovered from the MCP server and successfully invokes one; changing the server URL changes the toolset with no code edit.

LAB 12 · PROCESS MAP

How Lab 12 Runs

1. **Read how McpToolset discovers tools from…**
   cat lab12/agent.py
2. **Run the StreamableHTTP MCP agent**
   lab12/agent.py
3. **Send a request that exercises one of the…**
4. **Compare the SSE variant**
   cat lab12/agent.py
5. **Explain when SSE is preferred over…**

THE POINT

LO3 / LO2 — integrate third-party tools through the Model Context Protocol.

LAB 12 · PROCEDURE

Connect External Tools with MCP — StreamableHTTP and SSE — Part 1/2

1. **Read how McpToolset discovers tools from the server URL**

```
$ cat lab12/agent.py
```

2. **Run the StreamableHTTP MCP agent and read the printed tool list**

```
$ uv run python lab12/agent.py
```

3. Send a request that exercises one of the discovered tools

4. **Compare the SSE variant and note that only the connection params differ**

```
$ cat lab12/agent.py
```

LAB 12 · PROCEDURE

Connect External Tools with MCP — StreamableHTTP and SSE — Part 2/2

5 Explain when SSE is preferred over StreamableHTTP

6 Point the MCP_SERVER_URL at a different MCP server and re-run

7 Confirm the agent's available tools change without any change to the agent code

LAB 12 · VERIFY

Connect External Tools with MCP — StreamableHTTP and SSE

✅ Expected result

The agent prints the tools discovered from the MCP server and successfully invokes one; changing the server URL changes the toolset with no code edit.

IF IT DOESN'T WORK

Nothing happens

Check the .env file is in the labs folder and the key has no quotes or spaces.

Auth or 401 error

Re-copy the API key from AI Studio; confirm GOOGLE_GENAI_USE_VERTEXAI=0.

ModuleNotFoundError

Run uv sync again, and prefix commands with uv run so the venv is used.

TOPIC RECAP

Recap — Build A Multi Agent App with Gemini ADK

- You can now: LO3 — implement session management so an agent remembers earlier turns.
- You can now: LO3 / LO4 — analyse the agent execution loop and evaluate its behaviour.
- You can now: LO3 — implement agent-to-agent delegation with sub_agents.
- You can now: LO3 — design a coordinator agent routing to specialised sub-agents.
- You can now: LO3 / LO2 — orchestrate a fixed multi-stage pipeline with SequentialAgent.
- You can now: LO3 / LO4 — implement and evaluate a safety guardrail on an agent.

03

TOPIC 03

Build Agentic AI RAG in Gemini ADK

RAG pipeline · loading & splitting · embeddings · vector stores · retrieval · grounded answers

TOPIC WEIGHTING 25%

Key Concepts — Build Agentic AI RAG in Gemini ADK

1. Retrieval Augmented Generation (RAG) grounds an LLM in your own documents, reducing hallucination and keeping answers current.

2. The RAG pipeline is: load → split → embed → store → retrieve → generate.

3. Loaders convert PDF, HTML, CSV, Word and web sources into a common document object.

4. Splitting breaks documents into chunks small enough to embed and retrieve precisely, with overlap to preserve context.

5. An embedding maps text to a dense vector so that semantically similar text sits close together in vector space.

TOPIC WEIGHTING 25%

Key Concepts — Build Agentic AI RAG in Gemini ADK (continued)

1. A vector database (Chroma, FAISS, Pinecone, Milvus) stores embeddings and answers nearest-neighbour similarity queries.

2. Similarity search returns the closest chunks; Maximum Marginal Relevance (MMR) trades a little similarity for diversity.

3. In ADK, retrieval is wrapped as a tool so the agent decides when to consult the knowledge base.

4. RAG performance is evaluated on retrieval quality, groundedness/faithfulness, answer relevance and latency.

WHAT YOU'LL DO

Hands-On Labs — Build Agentic AI RAG in Gemini ADK

Lab 13

- Load, Split and Embed Documents into a Vector Store

Lab 14

- Build the Agentic RAG Agent — Retrieval as a Tool

Lab 15

- Evaluate RAG Performance — Retrieval Quality and Groundedness

TOPIC 03 · HANDS-ON

Load, Split and Embed Documents into a Vector Store

LAB 13

LO4 — build the ingestion half of a RAG pipeline and inspect the embeddings.

Take two product PDFs, extract their text, split it into retrievable chunks with page metadata, embed them and persist the vectors in a local Chroma database.

YOU'LL BUILD

A persistent Chroma collection holding the chunked and embedded air-fryer product and warranty manuals.

TOOLCHAIN

ChromaDB, pypdf, google-adk, Gemini embeddings

DONE WHEN

lab13/chroma_db exists and contains a populated air_fryer_docs collection, and you can state how the chunking rule changed the number of stored chunks.

LAB 13 · PROCESS MAP

How Lab 13 Runs

1. **Inspect the two source PDFs the agent will…**
   `ls lab13/*.pdf`
2. **Read the extraction function**
   `cat lab13/agent.py`
3. **Identify the chunking rule**
4. **Run the agent once to trigger ingestion…**
   `adk run lab13`
5. **Confirm the persistent vector store was…**
   `ls lab13/chroma_db`

THE POINT

LO4 — build the ingestion half of a RAG pipeline and inspect the embeddings.

LAB 13 · PROCEDURE

Load, Split and Embed Documents into a Vector Store — Part 1/2

1. **Inspect the two source PDFs the agent will be grounded in**

```
$ ls lab13/*.pdf
```

2. **Read the extraction function and note the page number kept as metadata**

```
$ cat lab13/agent.py
```

3. Identify the chunking rule and the minimum chunk length filter

4. **Run the agent once to trigger ingestion into ChromaDB**

```
$ uv run adk run lab13
```

LAB 13 · PROCEDURE

Load, Split and Embed Documents into a Vector Store — Part 2/2

5 **Confirm the persistent vector store was created on disk**

```
$ ls lab13/chroma_db
```

6 Explain why chunks shorter than 50 characters are discarded

7 **Change the chunk rule to split on single newlines, delete chroma_db, and re-ingest**

```
$ rm -rf lab13/chroma_db
```

8 Compare the resulting chunk count and note the effect on retrieval precision

LAB 13 · VERIFY

Load, Split and Embed Documents into a Vector Store

✅ Expected result

lab13/chroma_db exists and contains a populated air_fryer_docs collection, and you can state how the chunking rule changed the number of stored chunks.

IF IT DOESN'T WORK

Nothing happens

Check the .env file is in the labs folder and the key has no quotes or spaces.

Auth or 401 error

Re-copy the API key from AI Studio; confirm GOOGLE_GENAI_USE_VERTEXAI=0.

ModuleNotFoundError

Run uv sync again, and prefix commands with uv run so the venv is used.

TOPIC 03 · HANDS-ON

Build the Agentic RAG Agent — Retrieval as a Tool

LAB 14

LO4 / LO3 — wrap retrieval as a tool so the agent grounds its answers in your documents.

Complete the RAG loop: expose the vector search as an ADK tool, let the agent decide when to retrieve, and require it to cite the source document and page.

YOU'LL BUILD

lab14 — a grounded question-answering agent over the air-fryer manuals with citations.

TOOLCHAIN

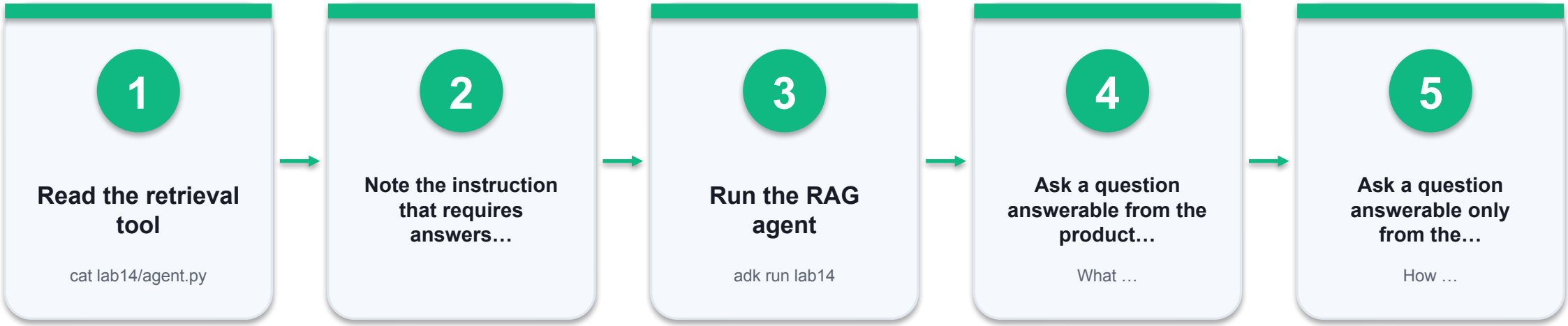
google-adk, ChromaDB, similarity search, Gemini 2.0 Flash

DONE WHEN

Both document questions are answered with the correct source and page cited, the out-of-scope question is refused rather than hallucinated, and you can see the retrieved chunks.

LAB 14 · PROCESS MAP

How Lab 14 Runs



THE POINT

LO4 / LO3 — wrap retrieval as a tool so the agent grounds its answers in your documents.

LAB 14 · PROCEDURE

Build the Agentic RAG Agent — Retrieval as a Tool — Part 1/2

1. **Read the retrieval tool and note how the query is embedded before searching**

```
$ cat lab14/agent.py
```

2. Note the instruction that requires answers to come only from retrieved context

3. **Run the RAG agent**

```
$ uv run adk run lab14
```

4. **Ask a question answerable from the product manual**

```
$ What temperature should I use to air fry chicken wings?
```

LAB 14 · PROCEDURE

Build the Agentic RAG Agent — Retrieval as a Tool — Part 2/2

5 **Ask a question answerable only from the warranty document**

```
$ How long is the warranty period and what does it exclude?
```

6 **Ask a question the documents do not cover and confirm the agent declines rather than invents**

```
$ What is the share price of the manufacturer?
```

7 **In adk web, inspect the retrieval function_response to see the chunks that were retrieved**

```
$ uv run adk web
```

8 Increase the number of retrieved chunks (n_results) and re-run the same questions

LAB 14 · VERIFY

Build the Agentic RAG Agent — Retrieval as a Tool

✅ Expected result

Both document questions are answered with the correct source and page cited, the out-of-scope question is refused rather than hallucinated, and you can see the retrieved chunks.

IF IT DOESN'T WORK

Nothing happens

Check the .env file is in the labs folder and the key has no quotes or spaces.

Auth or 401 error

Re-copy the API key from AI Studio; confirm GOOGLE_GENAI_USE_VERTEXAI=0.

ModuleNotFoundError

Run uv sync again, and prefix commands with uv run so the venv is used.

TOPIC 03 · HANDS-ON

Evaluate RAG Performance — Retrieval Quality and Groundedness

LAB 15

LO4 — evaluate the performance effectiveness of a RAG implementation.

Build a small evaluation set and score your RAG agent on retrieval hit rate, groundedness, answer relevance and latency, then tune one parameter and measure the improvement.

YOU'LL BUILD

A completed RAG evaluation table with a before-and-after comparison for one tuned parameter.

TOOLCHAIN

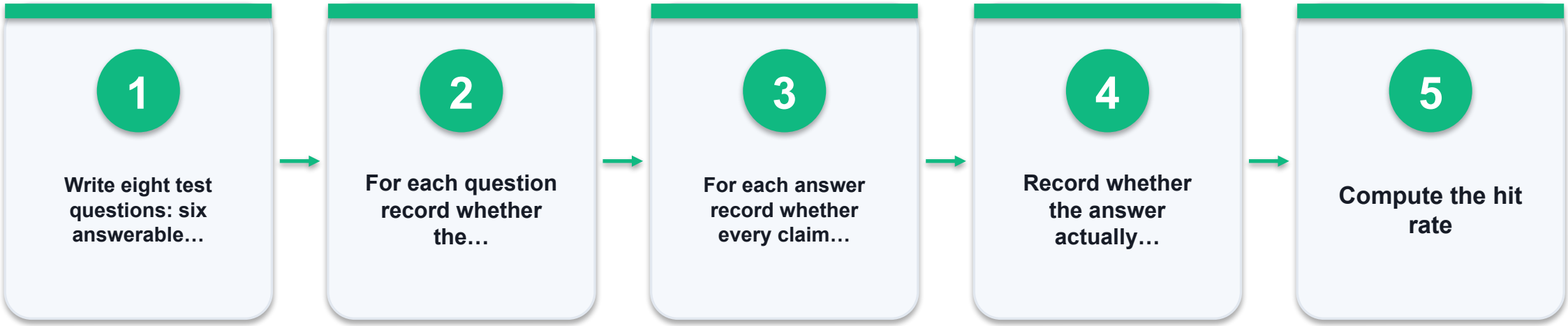
lab15, ChromaDB, similarity search vs MMR

DONE WHEN

A completed evaluation table with before-and-after figures for retrieval hit rate, groundedness and latency, plus a written, evidence-based tuning recommendation.

LAB 15 · PROCESS MAP

How Lab 15 Runs



THE POINT

LO4 — evaluate the performance effectiveness of a RAG implementation.

LAB 15 · PROCEDURE

Evaluate RAG Performance — Retrieval Quality and Groundedness — Part 1/2

1. Write eight test questions: six answerable from the PDFs, two deliberately out of scope
2. For each question record whether the correct chunk was retrieved (retrieval hit)
3. For each answer record whether every claim is supported by the retrieved text (groundedness)
4. Record whether the answer actually addresses the question (relevance) and its response time

LAB 15 · PROCEDURE

Evaluate RAG Performance — Retrieval Quality and Groundedness — Part 2/2

5 Compute the hit rate, groundedness rate and mean latency across the eight questions

6 Tune one parameter — chunk size, n_results, or switch similarity search to MMR

7 Re-run the same eight questions and recompute all three metrics

8 State which parameter change you would keep and justify it with your numbers

LAB 15 · VERIFY

Evaluate RAG Performance — Retrieval Quality and Groundedness

✅ Expected result

A completed evaluation table with before-and-after figures for retrieval hit rate, groundedness and latency, plus a written, evidence-based tuning recommendation.

IF IT DOESN'T WORK

Nothing happens

Check the .env file is in the labs folder and the key has no quotes or spaces.

Auth or 401 error

Re-copy the API key from AI Studio; confirm GOOGLE_GENAI_USE_VERTEXAI=0.

ModuleNotFoundError

Run uv sync again, and prefix commands with uv run so the venv is used.

TOPIC RECAP

Recap — Build Agentic AI RAG in Gemini ADK

- You can now: LO4 — build the ingestion half of a RAG pipeline and inspect the embeddings.
- You can now: LO4 / LO3 — wrap retrieval as a tool so the agent grounds its answers in your documents.
- You can now: LO4 — evaluate the performance effectiveness of a RAG implementation.

04

TOPIC 04

Build an Agentic AI App with Gemini Agent ADK and Streamlit

From agent to product · Streamlit chat UI · async Runner · deployment & feasibility

TOPIC WEIGHTING 15%

Key Concepts — Build an Agentic AI App with Gemini Agent ADK and Streamlit

1. A production agent app needs a user interface, session handling, error handling and API-key management.

2. Streamlit turns a Python script into a web app, making it the fastest route from an ADK agent to a usable product.

3. st.session_state persists the ADK SessionService and the chat history across Streamlit re-runs.

TOPIC WEIGHTING 15%

Key Concepts — Build an Agentic AI App with Gemini Agent ADK and Streamlit (continued)

1. The ADK Runner is async, so a Streamlit app drives it through asyncio.run() inside the request handler.

2. Secrets belong in .env and never in source control or in the repository you push to GitHub.

3. Assessing feasibility means weighing accuracy, latency, token cost, maintainability and governance against the business benefit.

WHAT YOU'LL DO

Hands-On Labs — Build an Agentic AI App with Gemini Agent ADK and Streamlit

Lab 16

- Declarative Agents — Configuring a Multi-Agent System in YAML

Lab 17

- Ship the Agent as a Web App with Streamlit

Lab 18

- Capstone — Design, Build and Assess Your Own Multi-Agent Application

TOPIC 04 · HANDS-ON

Declarative Agents — Configuring a Multi-Agent System in YAML

LAB 16

LO2 / LO3 — separate agent configuration from code to improve maintainability.

Define a five-specialist transport assistant entirely in YAML config files, with no Python agent code, and assess what this buys you for maintainability and governance.

YOU'LL BUILD

lab16 — a root agent with MRT, bus, taxi, bike and walk specialists, all declared in YAML.

TOOLCHAIN

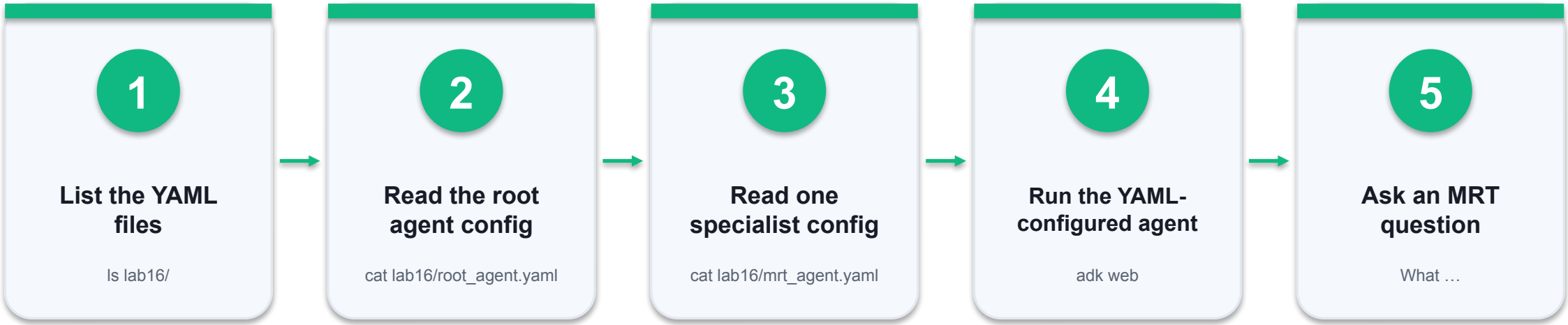
google-adk YAML config, LlmAgent, Gemini 2.5 Flash

DONE WHEN

All five specialists route correctly, and your sixth agent works after editing YAML only — no Python file was modified.

LAB 16 · PROCESS MAP

How Lab 16 Runs



THE POINT

LO2 / LO3 — separate agent configuration from code to improve maintainability.

LAB 16 · PROCEDURE

Declarative Agents — Configuring a Multi-Agent System in YAML — Part 1/2

1

List the YAML files and note one file per agent

```
$ ls lab16/
```

2

Read the root agent config and its sub_agents config_path list

```
$ cat lab16/root_agent.yaml
```

3

Read one specialist config and compare its fields to the Python Agent(...) arguments

```
$ cat lab16/mrt_agent.yaml
```

4

Run the YAML-configured agent

```
$ uv run adk web
```

LAB 16 · PROCEDURE

Declarative Agents — Configuring a Multi-Agent System in YAML — Part 2/2

5 **Ask an MRT question and confirm it routes to mrt_agent**

```
$ What is the fastest MRT route from Bugis to Woodlands?
```

6 Ask a cycling question and confirm it routes to bike_agent

7 Add a sixth specialist by creating a new YAML file and referencing it in root_agent.yaml

8 Re-run and confirm the new specialist is routed to without touching any Python file

LAB 16 · VERIFY

Declarative Agents — Configuring a Multi-Agent System in YAML

✅ Expected result

All five specialists route correctly, and your sixth agent works after editing YAML only — no Python file was modified.

IF IT DOESN'T WORK

Nothing happens

Check the .env file is in the labs folder and the key has no quotes or spaces.

Auth or 401 error

Re-copy the API key from AI Studio; confirm GOOGLE_GENAI_USE_VERTEXAI=0.

ModuleNotFoundError

Run uv sync again, and prefix commands with uv run so the venv is used.

TOPIC 04 · HANDS-ON

Ship the Agent as a Web App with Streamlit

LAB 17

LO3 — deploy a multi-agent system behind a usable chat interface.

Wrap the sequential transport agent in a Streamlit chat UI, wiring the async ADK Runner and persisting the session across Streamlit re-runs so the conversation survives.

YOU'LL BUILD

lab17 — a browser chat application backed by the multi-agent transport workflow.

TOOLCHAIN

Streamlit, google-adk Runner, InMemorySessionService, asyncio

DONE WHEN

The Streamlit app answers a journey query, retains history across follow-up turns, and your Clear chat button empties the conversation without restarting the server.

LAB 17 · PROCESS MAP

How Lab 17 Runs

1

Read how the ADK session service is stored…

cat lab17/app.py

2

Note that the async Runner is driven…

3

Launch the Streamlit application

streamlit run lab17/app.py

4

Open the app in the browser

From Tampines to Orchard…

5

Send a follow-up question

What if I cycle instead?

THE POINT

LO3 — deploy a multi-agent system behind a usable chat interface.

LAB 17 · PROCEDURE

Ship the Agent as a Web App with Streamlit — Part 1/2

1. **Read how the ADK session service is stored in st.session_state**

```
$ cat lab17/app.py
```

2. Note that the async Runner is driven through asyncio.run inside the handler

3. **Launch the Streamlit application**

```
$ uv run streamlit run lab17/app.py
```

4. **Open the app in the browser and submit a journey**

```
$ From Tampines to Orchard Road
```

LAB 17 · PROCEDURE

Ship the Agent as a Web App with Streamlit — Part 2/2

5 **Send a follow-up question and confirm the conversation history is retained**

```
$ What if I cycle instead?
```

6 Refresh the browser and observe what happens to the session

7 Change the page title and icon in st.set_page_config and reload

8 Add a sidebar Clear chat button that resets st.session_state.messages

LAB 17 · VERIFY

Ship the Agent as a Web App with Streamlit

✅ Expected result

The Streamlit app answers a journey query, retains history across follow-up turns, and your Clear chat button empties the conversation without restarting the server.

IF IT DOESN'T WORK

Nothing happens

Check the .env file is in the labs folder and the key has no quotes or spaces.

Auth or 401 error

Re-copy the API key from AI Studio; confirm GOOGLE_GENAI_USE_VERTEXAI=0.

ModuleNotFoundError

Run uv sync again, and prefix commands with uv run so the venv is used.

TOPIC 04 · HANDS-ON

Capstone — Design, Build and Assess Your Own Multi-Agent Application

LAB 18

LO1 / LO2 / LO3 / LO4 — design a multi-agent solution and assess its feasibility.

In groups of three to five, design and build a multi-agent ADK application for a use case in your own industry, then present it with a feasibility assessment.

YOU'LL BUILD

A working multi-agent application with at least three agents, at least two tools, session memory and a documented feasibility assessment.

TOOLCHAIN

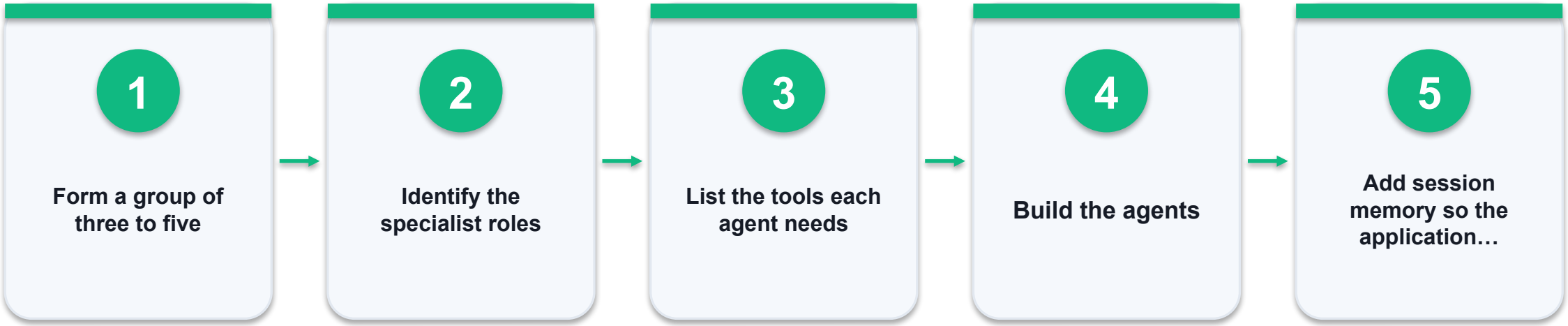
google-adk, sub_agents or SequentialAgent, custom tools, optional RAG and Streamlit

DONE WHEN

A running application with three or more agents and two or more tools, demonstrated live, plus a feasibility assessment stating whether you would recommend production deployment and on what evidence.

LAB 18 · PROCESS MAP

How Lab 18 Runs



THE POINT

LO1 / LO2 / LO3 / LO4 — design a multi-agent solution and assess its feasibility.

LAB 18 · PROCEDURE

Capstone — Design, Build and Assess Your Own Multi-Agent Application — Part 1/3

1. Form a group of three to five and choose an industrial use case from your own sector
2. Identify the specialist roles and draw the agent topology — coordinator or sequential
3. List the tools each agent needs and which are custom functions versus built-in
4. Build the agents, starting from the closest lab in this course as your template

LAB 18 · PROCEDURE

Capstone — Design, Build and Assess Your Own Multi-Agent Application — Part 2/3

5. Add session memory so the application handles multi-turn conversations

6. Add a guardrail or a structured output schema appropriate to your use case

7. Test with at least five realistic prompts and record where the agent fails

8. Assess feasibility: accuracy, latency, token cost, maintainability and governance risk

LAB 18 · PROCEDURE

Capstone — Design, Build and Assess Your Own Multi-Agent Application — Part 3/3

9 Present the application and the feasibility assessment to the class in five minutes

LAB 18 · VERIFY

Capstone — Design, Build and Assess Your Own Multi-Agent Application

✅ Expected result

A running application with three or more agents and two or more tools, demonstrated live, plus a feasibility assessment stating whether you would recommend production deployment and on what evidence.

IF IT DOESN'T WORK

Nothing happens

Check the .env file is in the labs folder and the key has no quotes or spaces.

Auth or 401 error

Re-copy the API key from AI Studio; confirm GOOGLE_GENAI_USE_VERTEXAI=0.

ModuleNotFoundError

Run uv sync again, and prefix commands with uv run so the venv is used.

TOPIC RECAP

Recap — Build an Agentic AI App with Gemini Agent ADK and Streamlit

- You can now: LO2 / LO3 — separate agent configuration from code to improve maintainability.
- You can now: LO3 — deploy a multi-agent system behind a usable chat interface.
- You can now: LO1 / LO2 / LO3 / LO4 — design a multi-agent solution and assess its feasibility.

WRAP-UP

Course Summary & Next Steps

LEARNING OUTCOMES

What You Achieved

1

LO1 · Analysed LLM applications

Explained how LLMs work and identified Generative AI use cases across finance, marketing, education, healthcare and legal.

2

LO2 · Established Gemini designs

Chose Gemini models and tuned parameters, and assessed the improvements agents bring to engineering processes.

3

LO3 · Developed LLM applications

Built single agents, tool-using agents, multi-agent handoff, sequential workflows, guardrails, structured output and MCP integrations.

4

LO4 · Evaluated RAG

Built an agentic RAG pipeline over your own PDFs and measured retrieval hit rate, groundedness and latency.

NEXT STEPS

Continue Your Learning

1

Official ADK docs

https://google.github.io/adk-docs/ — the reference for every ADK class and pattern.

2

Google AI Studio

https://aistudio.google.com — prototype prompts and manage your API keys.

3

Rebuild the labs

Redo each lab from a blank file until the agent pattern is automatic.

4

Extend the capstone

Add RAG, a guardrail and a Streamlit UI to your own multi-agent application.

5

Deploy it

Take your agent to Cloud Run or Vertex AI Agent Engine for production use.

6

Share it

Publish your agent to GitHub and gather feedback from real users.

CONTINUE WITH TERTIARY INFOTECH

Recommended Courses

1. WSQ - Develop Generative AI Solutions with Azure OpenAI Service
2. WSQ - Mastering Prompt Engineering for Generative AI Content Creation
3. WSQ - Build LLM Applications Using Langchain and Flowise
4. WSQ - Digital Transformation and Business Innovation with Generative AI (GenAI)
5. WSQ - Practical Image Generation Using GAN, VAE, and Diffusion Models

WE'RE HERE TO HELP

Support

- If you have any enquiries during or after the class, you can contact us below.
- Email: enquiry@tertiaryinfotech.com
- Tel: +65 6100 0613
- Website: www.tertiarycourses.com.sg
- LMS / TMS: https://lms-tms.tertiaryinfotech.com

WRAP-UP

Assessment

1

Written Assessment (SAQ)

1 hour · open book · short-answer knowledge questions.

2

Practical Performance (PP)

1 hour · open book · hands-on Gemini ADK agent build tasks.

3

Digital attendance

Remember to take the Assessment digital attendance (TRAQOM) before you start.

4

Submit on the LMS

Upload your completed answers at https://lms-tms.tertiaryinfotech.com/

ON ASSESSMENT DAY

Assessment Flow

1

TRAQOM survey

Scan the QR code on the…

2

Digital attendance

Scan the SSG assessment QR

3

Sit WA then PP

Open book · 1 hour each

4

Submit on the LMS

Upload your completed…

5

Sign the record

Sign the Assessment…

REMEMBER

All five steps are mandatory for WSQ funding — missing the digital attendance or the TRAQOM survey can invalidate your claim.

Digital Attendance (Mandatory)

1

Three times a day

Take the AM, PM and Assessment digital attendance — mandatory for every WSQ-funded course.

2

Trainer shows the QR

The trainer or administrator displays the digital attendance QR code from the SSG portal.

3

Scan and submit

Scan the QR code with your mobile phone camera and submit your attendance.

4

75% minimum

A minimum of 75% attendance is required to be eligible for assessment and funding.

HAPPY BUILDING

Thank You!

You can now design, build and evaluate multi-agent AI applications with the Gemini Agent Development Kit.